ChatGPT

Core CS for SWE Hiring in India

Executive summary

For general SWE/SDE hiring in India, **Data Structures and Algorithms remains the single biggest gatekeeper**, especially at the online assessment, phone-screen, and coding-round stages across Amazon, Google, Microsoft, Flipkart, Swiggy, and startup/product-company loops. Official interview-prep material from Amazon, Microsoft, and Flipkart all foreground coding, algorithms, problem solving, object-oriented design, and design/system thinking; recent candidate reports from India and India-targeted teams show the same pattern. [1]

Operating Systems matters, but usually as a tier-two subject rather than the first gate for mainstream SWE roles. In most entry-level and early-career generalist hiring, OS is rarely the decisive factor in resume screening or coding OAs. Its value shows up later: technical interviews, project deep dives, machine-coding/OOD rounds, and system-design or backend/platform discussions. OS becomes much more important for backend/platform/infrastructure/support roles, and for teams that explicitly mention distributed systems, performance, scalability, or production operations. Amazon's India-linked job postings explicitly call out OS, OOD, algorithms, and data structures; Flipkart's own prep guide says OS basics are essential for system design; Microsoft emphasizes distributed systems and design; startup/product roles in India increasingly ask for large-scale distributed systems experience. [2]

The **practical ranking for most candidates with a CS background targeting general SWE/SDE roles in India** is:

1. **DSA**
2. **OOP/OOD**
3. **DBMS + SQL**
4. **Operating Systems**
5. **Computer Networks**
6. **System Design**
7. **Concurrency / Multithreading**
8. **SDLC / testing / debugging / project depth**
9. **Compilers** for specialized roles only

That ranking changes for **mid-level backend/platform roles**: system design and distributed systems jump near the top, and OS/networking/concurrency become more important because they are embedded inside design, performance, and debugging discussions. Google candidate reports from India show L4/L5 loops that are heavily DSA-driven but include system design depending on level and interview structure; Microsoft's official guidance similarly weights design and distributed systems strongly for applicable roles. [3]

For **Indian IT-services fresher roles** such as TCS, Infosys, and Wipro, the picture is different. The main filter is often **aptitude, coding/pseudocode, and communication**, with core CS subjects asked selectively in

technical interviews rather than dominating the entire process. Recent TCS/Infosys/Wipro process summaries show TCS NQT centered on aptitude and coding, Infosys assessment emphasizing reasoning, pseudocode, English, and puzzle/problem sections, and Wipro combining aptitude, coding, and a lighter technical interview that often asks OS, DBMS, and OOP basics rather than deep system design. [4]

From a time-efficiency perspective, the best fast-learning stack for a CS student is: **NeetCode or Striver for DSA, OSTEP selected chapters for OS, SQLBolt plus selective DB lectures for DBMS, a selective networking course for CN, Refactoring.Guru for OOD/design patterns, and structured mock/problem practice**. For most India-focused interview prep, an **8-week plan** is the best tradeoff between speed and retention; a **4-week plan** works only if you restrict scope to the minimum interview-relevant topics and practice aggressively. [5]

Market context and method

India's software hiring market in 2025–2026 has been **selective rather than uniformly weak**. TeamLease EdTech reported that **70% of employers intended to hire freshers** in July–December 2025, with **e-commerce and technology startups at 88%** and **Bangalore leading major cities at 81%**. The same report describes a shift toward **job-ready, digitally skilled freshers** rather than purely volume hiring. Naukri's 2026 JobSpeak summaries show white-collar hiring growth with AI/ML demand notably stronger than general IT, while Reuters and GCC-focused reporting indicate continued expansion of global capability centers and multinational engineering presence in India. [6]

That market context matters because it changes **what gets screened**. In a precision-hiring environment, companies are not usually testing "academic completeness"; they are testing whether you can clear the **specific bottleneck of their process**. For TCS/Infosys/Wipro, that bottleneck is often the assessment plus communication. For Amazon/Google/Microsoft and top Indian product companies, it is still typically coding/problem solving first, then design/project depth. Official company guidance strongly supports that interpretation. Amazon's university and SDE interview pages emphasize coding, algorithms, OOD, databases, distributed computing, OS, and system design; Microsoft's official guidance evaluates problem solving, design, coding, testing, and—in relevant roles—distributed systems; Flipkart's own guide separates problem-solving/data-structure and design rounds and explicitly recommends brushing up OOP, design patterns, system architecture, distributed systems, Internet basics, and OS basics. [7]

This report uses four evidence buckets from roughly the last two to three years: **official interview-prep pages and job postings**, **recent candidate experiences** from LeetCode/Medium/GeeksforGeeks, **recruiter/career guidance from company career pages**, and **India hiring reports**. Where public company-specific evidence was sparse, I grouped firms into clusters instead of pretending there is a precise company-by-company scoring rubric. In particular, **Ola and several smaller startups had limited recent accessible public process evidence**, so they are treated inside an Indian product/startup cluster rather than given a false precision score. The weight tables below are therefore **analytical estimates**, not official hiring rubrics. [8]

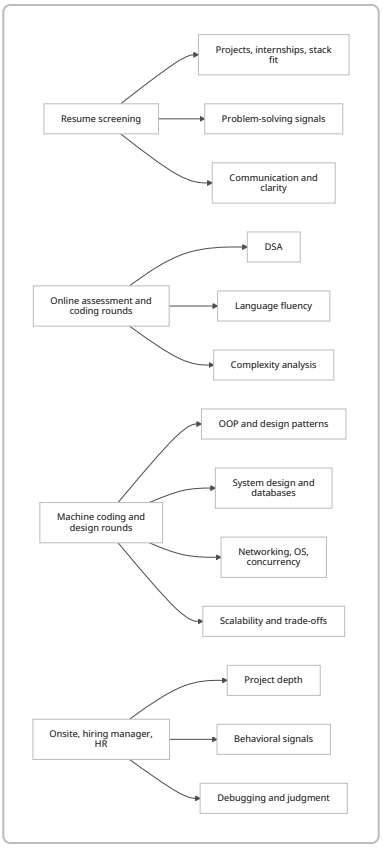
A second methodological point: **resume screening is not very "subject-driven" in the same way coding rounds are**. Resumes are screened more through signals like internships, known brands, project depth, open-source work, contest/problem-solving profiles, and stack fit. Microsoft candidate advice explicitly mentions keeping LinkedIn, resume, projects, internships, and coding profiles updated; Flipkart's exploratory and phone-screen stages focus heavily on prior work and project depth; Google candidate

reports similarly stress visibility, experience, and project quality before interview performance even becomes relevant. [9]

Hiring-stage importance by company cluster

The most reliable way to answer “How important are OS and core CS subjects?” is to separate **company clusters** and **career stages**.

What the stages usually screen for



This stage map reflects the official process descriptions from Amazon, Microsoft, and Flipkart, along with recent India-centered candidate reports from Amazon, Google, Microsoft, Flipkart, and Swiggy. [10]

Stage emphasis by cluster

The percentages below are **estimated stage importance** for success, not official score breakdowns.

Company cluster	Resume screening	OA / coding rounds	Design / machine coding / technical deep dive	Behavioral / HM / HR
TCS / Infosys / Wipro style fresher hiring	10%	50%	15%	25%
Indian product / startup cluster in India	15%	35%	30%	20%
Global big tech India, early-career SWE	15%	45%	30%	10%
Global big tech India, mid-level backend/ platform	10%	25%	45%	20%

These estimates are synthesized from TCS NQT/PrepInsta pattern updates, recent Infosys and Wipro process summaries, Amazon official OA and interview-loop descriptions, Microsoft's official technical interview guidance, Flipkart's official SDE prep guide, and recent India interview reports from LeetCode/ Medium/GFG. [11]

Core-subject importance for Indian IT-services fresher hiring

Scale: **5 = primary gate**, **4 = major**, **3 = regular**, **2 = occasional**, **1 = rare**, **0 = generally irrelevant**.

Subject	Resume	OA / coding	Technical interview	HR / HM
DSA / pseudocode / basic problem solving	1	4	2	1
OOP / SDLC basics	1	1	2	1
DBMS / SQL	0	1	2	1
OS	0	1	2	1
Networking	0	0–1	1	1
System design	0	0	0–1	0
Concurrency	0	0	1	0
Compilers	0	0	0	0

Why this is the right interpretation: TCS NQT is structured around aptitude and coding; Infosys assessment heavily emphasizes reasoning, English, puzzle/managing sectional cutoffs, and pseudocode, while recent candidate experience says the interview is often more managerial than deeply technical; Wipro's recent process summaries still show aptitude plus basic coding as the major gate, with technical interviews commonly limited to OS/DBMS/OOP basics rather than deep systems design. [12]

Core-subject importance for Indian product companies and startups

This cluster covers companies such as **Flipkart, Swiggy, Postman Bengaluru/Hyderabad, Razorpay-type startups, and similar product firms**. It also covers **Ola-like firms** where recent public process detail was sparse but role shape is similar.

Subject	Resume	OA / coding	Machine coding / design	Behavioral / project deep dive
DSA	3	5	2	2
OOP / OOD	2	2–3	5	2
DBMS / SQL	1	1–2	3	2
OS	1	1	3	2
Networking	1	1	3	1
System design	1	1	4	2
Concurrency / multithreading	1	1	3	2
Compilers	0	0	0–1	0

Flipkart's official prep guide is especially revealing: problem solving/DSA gets its own round; design gets its own round; OOP, design patterns, distributed systems, CAP, cache and queue systems, Internet basics, and **OS basics are called out as essential for the system-design round**. Recent Swiggy experiences show DSA-heavy online tests and system-design emphasis for SDE-I/SDE-II back-end processes. Postman's Bengaluru roles explicitly seek distributed-systems and large-scale platform/backend engineering experience, which pushes OS/networking/concurrency from "nice to know" into "useful in real interviews." [13]

Core-subject importance for global big tech hiring in India

This table fits **Amazon India, Google India, Microsoft India**, plus comparable large GCC engineering teams hiring in India.

Subject	Resume	OA / coding	Technical / design loop	Behavioral / project deep dive
DSA	3	5	4	2
OOP / OOD	2	2–3	4	2
System design / distributed systems	1 for entry, 3 for 2+ YOE	1–2	3 for entry, 5 for 2+ YOE	2
DBMS / SQL	1–2	1–2	3	2
OS	1–2	1	2–3	2

Subject	Resume	OA / coding	Technical / design loop	Behavioral / project deep dive
Networking	1	1	2–3	1
Concurrency	1	1–2	3	2
Compilers	0	0	0–1 general, 5 specialized	0

Why the split matters:

Amazon's official interview-topics page explicitly lists **data structures, algorithms, coding, object-oriented design, databases, distributed computing, and operating systems**; its SDE-II OA includes **coding + system design + work style**; and India-linked job postings repeatedly mention **OOD, OS, DSA, databases, scale, and distributed systems**. Microsoft officially evaluates **problem solving, design, coding, testing**, and says distributed systems knowledge is very important where relevant. Google job postings still foreground **DSA and software design**, while Google candidate reports from India show **DSA-dominant loops with system design added depending on level/packet**. [14]

Bottom line on OS

For **general SWE roles**, OS is usually **not more important than DSA, OOP/OOD, DBMS, or system design**. But it is still important enough that skipping it entirely is a mistake.

A good rule of thumb is:

- **TCS / Infosys / Wipro general fresher roles**: OS is **moderately useful**, mostly for technical interviews, and rarely a first-stage gate. [15]
- **Amazon / Microsoft / Google general SWE**: OS is **secondary but real**; it helps in team-fit, project deep dive, technical follow-ups, and any systems-oriented discussion. [16]
- **Flipkart / Swiggy / startups / backend-platform roles**: OS is **more important than many students assume**, because it appears disguised inside queues, caches, concurrency, latency, retries, locking, resource contention, and machine-coding design. [17]
- **Compiler / kernel / infra / networking / systems roles**: OS can become **top-tier**, and compilers/networking can also jump from low-priority to critical. Google's TPU Compiler role is an example of a specialist posting where compiler- and systems-oriented depth matters far more than for mainstream SWE roles. [18]

How important OS and core CS really are

The simplest defensible answer is this: **OS is not the main gate; it is the multiplier**.

DSA gets you through the door because it dominates OAs, live coding, and early technical filtering. That is overwhelmingly clear from Amazon's official OA structure, Microsoft's coding-first advice, Flipkart's PS/DS round, and recent Amazon/Google/Microsoft/Flipkart/Swiggy reports. [19]

OS then acts as a multiplier in three situations.

First, it helps when you are asked to reason about **real production behavior**: why a service slows down, why threads deadlock, why performance tanks under contention, how memory pressure changes behavior, why a file write is not durable immediately, or why containerized systems differ from full VMs. This is why official prep material from Flipkart and Amazon connects OS to design and systems discussions, not just textbook viva questions. [20]

Second, it strengthens **machine coding and OOD**. Many companies do not ask "Explain page replacement" in isolation; instead they ask for a design—parking lot, Splitwise, chess game, notification platform, distributed cache—and then evaluate whether you understand thread safety, state, resource ownership, lifecycle, and failure modes. Recent Amazon fresher and Microsoft practice accounts show exactly this combination of DSA plus machine-coding/OOD/constraint-follow-ups. [21]

Third, OS becomes valuable in **project deep dives and HM rounds** because interviewers trust evidence of applied understanding more than memorized definitions. Flipkart's phone screen and exploratory stage, Microsoft's official guidance, and Google candidate reports all make clear that interviewers probe your prior projects and technical choices. If your résumé contains backend, systems, cloud, databases, web servers, compilers, or any performance-sensitive system, OS knowledge becomes part of how believable your project explanations sound. [22]

For a student or early-career engineer with limited time, the implication is very practical: **learn OS to interview depth, not to full-academic depth**. That means understanding the concepts and being able to apply them in explanations, trade-offs, and simple design/problem contexts. You do not need a full graduate-level systems course for most interview loops. You do need enough depth to answer confidently and connect OS ideas to code and architecture. [23]

OS topic map and question patterns

Interview-relevant OS topics

OS topic	Typical question forms	Where it appears most	Typical difficulty
Processes vs threads	Compare process/thread, context switch cost, when to use which	Technical interview, project deep dive	Easy
Scheduling	FCFS/RR/SJF, throughput vs latency, starvation vs fairness	Core-CS viva, systems discussion	Easy–Moderate
Virtual memory	Paging, TLB, page fault, fragmentation, locality	Technical interview, systems/design follow-up	Moderate
Memory management	Stack vs heap, malloc/free, leaks, GC trade-offs	Language + OS crossover, project deep dive	Easy–Moderate

OS topic	Typical question forms	Where it appears most	Typical difficulty
Synchronization	Mutex vs semaphore vs condition variable, producer-consumer, race conditions	OS technical round, machine coding, backend	Moderate
Deadlocks	Necessary conditions, avoidance, prevention, detection	Service-firm technical rounds, backend/ system roles	Easy–Moderate
File systems / durability	Inodes, journaling, fsync, append/ logging, crash consistency	System design, storage-heavy roles	Moderate
Virtualization / containers	VM vs container, isolation, namespaces/cgroups, overhead	DevOps/platform/ backend/system design	Moderate
Concurrency / multithreading	Thread safety, lock granularity, contention, parallelism	Machine coding, OOD, backend/platform roles	Moderate–Hard

This table is aligned with OSTEP's structure around virtualization, concurrency, and persistence, with company evidence showing where these concepts surface in hiring. Amazon explicitly lists OS among software-development topics; Wipro's recent process summaries mention deadlocks and semaphores; Flipkart's guide links OS basics to system design; startup/product roles in India increasingly emphasize distributed systems and platform behavior. [24]

Other core CS topics mapped to interview behavior

Subject	High-frequency question types	Most likely stage	Typical difficulty
DSA	Arrays, strings, trees, graphs, heaps, DP, backtracking, complexity	OA, phone screen, live coding	Moderate–Hard
DBMS / SQL	JOINS, normalization, ACID, indexing, transactions, isolation	Technical round, project deep dive, design	Easy–Moderate
Networking	HTTP/TCP/DNS/load balancer/retries/ timeouts/CAP-related trade-offs	System design, backend, project discussion	Easy–Moderate
OOP / OOD	SOLID, class modeling, patterns, extensibility, maintainability	Machine coding, design, technical loop	Moderate
System design	Caching, queues, partitioning, replication, rate limiting, trade-offs	Design rounds, mid-level roles	Moderate–Hard
Concurrency	Shared state, synchronization, queues, worker pools, race conditions	Machine coding, backend/ platform/system design	Moderate–Hard
Compilers	Lexing/parsing, AST, optimization, low-level transformation	Specialized compiler/tools roles	Hard but rare

The evidence base is unusually consistent here. Flipkart explicitly names DSA, OOD, design patterns, system architecture, CAP, distributed systems, and OS/Internet basics. Amazon lists DSA, OOD, databases, distributed computing, and OS. Microsoft's official guidance adds testing and distributed systems. Recent technical experiences from Microsoft, TCS, Wipro, and Infosys show DBMS, OOP, OS, and SDLC/Agile concepts appearing most often as **supporting technical knowledge**, not as the central first-round gate. [25]

Fastest interview-resilient OS coverage

If your goal is interview efficiency, prioritize this OS subset:

- processes vs threads
- context switching
- scheduling basics
- synchronization primitives
- deadlocks and starvation
- virtual memory, paging, TLB, page faults
- heap vs stack, fragmentation
- file-system basics and durability intuition
- VM vs container
- one or two real examples of concurrency bugs in your own code

That subset covers the majority of OS-style questions you are likely to face in general SWE/SDE interviews in India. It also maps well into design reasoning for backend, platform, or machine-coding rounds. [26]

Study plans and minimal checklists

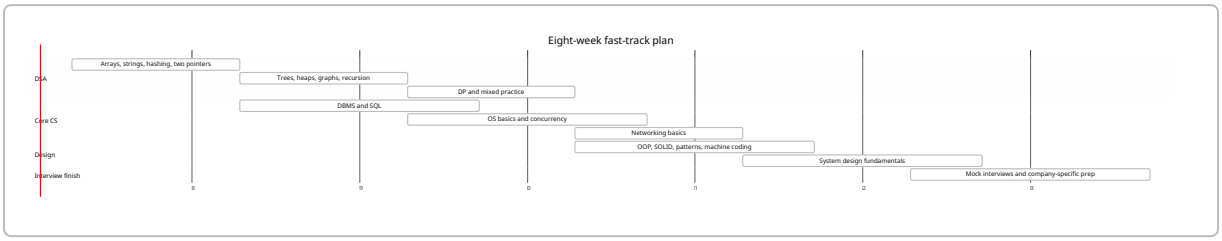
Recommended time budgets

Assuming you already have basic CS exposure:

- **4-week crash plan**: about **3–4 hours/day on weekdays**, **5–6 hours/day on weekends**
- **8-week balanced plan**: about **2–3 hours/day on weekdays**, **4–5 hours/day on weekends**
- **12-week strong plan**: about **1.5–2.5 hours/day consistently**

The 4-week option is enough to become interview-viable for many entry-level loops. The 8-week option is the best default if you want to cover DSA plus enough OS/DBMS/CN/OOD to sound credible in technical discussions. The 12-week option is the best choice if you are targeting product companies, startups, or 2+ YOE backend/platform roles where system design is likely. This recommendation follows directly from the relative interview weight of DSA first, then OOD/design, then core-CS support topics. [27]

Timeline visual



The plan structure mirrors what current India hiring loops actually reward: DSA first, then OOD/design, then core-CS reinforcement, then mock execution. [28]

Four-week crash plan

Week one

Focus almost entirely on DSA foundations and SQL basics. Finish arrays, strings, hashing, sliding window, two pointers, stack/queue, and binary search. Add 30–45 minutes per day of SQL or DBMS basics so that joins, group by, normalization, and ACID do not feel foreign. The goal is to make yourself OA-ready immediately. This is the highest-return week because OAs and first technical rounds are still overwhelmingly DSA-heavy. [29]

Week two

Cover trees, heaps, graphs, BFS/DFS, and basic dynamic programming. Start OS with processes vs threads, scheduling, synchronization, deadlocks, and virtual memory basics. At the end of the week, do one mock technical round with a friend or by recording yourself. This is where you convert from “practice mode” to “interview mode.” [30]

Week three

Finish DBMS essentials, networking basics, and OOP/OOD essentials. Learn SOLID, common design patterns, class modeling, and one or two machine-coding classics such as Splitwise, parking lot, chess, or elevator. This maps directly to Flipkart-style design/OOD rounds and the machine-coding/OOD flavor that increasingly appears in Amazon/startup/product loops. [31]

Week four

Run mixed interview simulations: two coding rounds, one OOD/machine-coding discussion, one system-design-lite round, one behavioral/project deep dive. Review your resume line by line and prepare stories for each project, especially anything involving databases, web servers, APIs, concurrency, or scaling. Companies repeatedly probe project depth and role fit at the later stages. [32]

Minimal four-week checklist

- [ ] Solve **60–80** quality DSA problems

- [ ] Be comfortable with **arrays, strings, hash maps, stacks, queues, trees, graphs, heaps, binary search, DFS/BFS, basic DP**
- [ ] Learn **joins, normalization, ACID, indexes, transactions**
- [ ] Learn **processes/threads, deadlocks, synchronization, virtual memory**
- [ ] Learn **HTTP, TCP basics, DNS, load balancer, cache, queue**
- [ ] Learn **SOLID + 5–8 design patterns**
- [ ] Practice **2 machine-coding/OOD problems**
- [ ] Practice **2 system-design-lite prompts**
- [ ] Prepare **5 STAR stories** for projects/conflict/ownership/failure

Eight-week balanced plan

Weeks one to three

Build DSA steadily and deeply enough that medium difficulty problems stop feeling unfamiliar. A good target is **90–120 problems**, but quality matters more than the number. Use a structured roadmap instead of random company tags. Recent Google India and Amazon reports both reinforce that success comes from pattern mastery and articulation, not just grinding random problems. [33]

Weeks four and five

Do OS, DBMS, and networking in parallel, but study them the way interviews ask them: compare/contrast, trade-offs, and production examples. Avoid spending time on low-yield theory that almost never appears in general SWE interviews. This is where most candidates can leap from “coding-only” to “well-rounded.” [34]

Week six

Do OOP/OOD, design patterns, and concurrency through small designs instead of passive reading. Parking lot, Splitwise, elevator, LRU cache, and rate limiter are all useful because they force you to combine OOD, state, data structures, and thread-safety reasoning. This is especially important for product/startup flips where depth, not memorization, matters. [35]

Week seven

Study system design fundamentals: availability vs consistency, caching, queues, partitioning, replication, retries, idempotency, data stores, and observability. You do not need full DDIA-level depth for most entry and junior interviews, but you do need enough to structure a design conversation. Amazon, Microsoft, Flipkart, and Google candidate evidence all show design becoming material by higher junior and mid-level roles. [36]

Week eight

Run mocks and tailor to target companies:

- TCS/Infosys/Wipro: aptitude/pseudocode/communication plus core technical basics
- Amazon/Microsoft/Google: coding plus project deep dives; add design if not strictly fresher
- Flipkart/Swiggy/startups: coding plus machine coding/OOD/system design

That company-specific tail-end specialization is one of the highest-ROI things you can do. [37]

Minimal eight-week checklist

- [ ] Solve **90–120** structured DSA problems
- [ ] Cover at least **one full roadmap** subset such as NeetCode 150 core sections
- [ ] Finish **interview-depth OS**: threads, locks, deadlocks, paging, virtual memory, file-system intuition
- [ ] Finish **interview-depth DBMS**: SQL, ACID, isolation, indexing, normalization
- [ ] Finish **interview-depth CN**: TCP, HTTP, DNS, proxies/load balancing, caching, retries/timeouts
- [ ] Do **3–4 machine-coding/OOD problems**
- [ ] Do **4–6 system-design-lite prompts**
- [ ] Run **4 mock technical interviews**
- [ ] Prepare your resume/project explanations until they sound natural, concise, and defensible

Twelve-week extension

Use weeks nine to twelve only if you are targeting **Google/Microsoft/Amazon mid-level**, **backend/platform roles**, or **higher-signal product companies**. That extension should mostly go into:

- harder graphs and DP
- distributed systems/system design
- concurrency/thread safety
- production debugging stories
- one visible project or open-source contribution that demonstrates system thinking

This is the phase where OS and networking stop being “theory revision” and become part of your engineering story. [38]

Best time-efficient resources

Resource comparison

The times below are **practical interview-prep estimates**, not official course durations.

Resource	Best for	Estimated time for interview prep	Depth	Cost	Format	Why it is efficient
NeetCode Roadmap / NeetCode 150	DSA interview prep	40–70 hours	Medium–High	Free / optional Pro	Structured problem set + videos	Gives ordered topic progression and curated interview problems instead of random grinding. [39]
Striver A2Z DSA	DSA from basics to advanced	50–80 hours if selective	High	Free	Article + sheet + videos	Especially good for India-based prep because it is structured and broad; use selectively, not exhaustively, if time is short. [40]
CSES Problem Set	Clean structured algorithm practice	20–40 hours selective	Medium–High	Free	Problem set	Great for disciplined practice after concept learning; high signal, low noise. [41]

Resource	Best for	Estimated time for interview prep	Depth	Cost	Format	Why it is efficient
OSTEP	Operating Systems	15–25 hours selected chapters	High	Free	Online book	Best interview-efficient OS resource because it is organized around virtualization, concurrency, persistence—the same buckets that matter in interviews. [42]
SQLBolt	Fast SQL/ DBMS basics	4–6 hours	Low–Medium	Free	Interactive browser lessons	Very fast way to get joins, aggregation, and SQL fluency before deeper DBMS review. [43]
CMU 15-445 selected lectures	DBMS internals and indexing/ transactions	15–25 hours selective	High	Free	University course	Best upgrade path after SQL basics if you want real understanding of indexes, storage, and transactions. [44]
Stanford CS144 selected lectures	Networking fundamentals	8–15 hours selective	Medium–High	Free	University course	Strong networking intuition for TCP/IP, Internet structure, and practical systems thinking. [45]

Resource	Best for	Estimated time for interview prep	Depth	Cost	Format	Why it is efficient
Refactoring.Guru	OOP, SOLID, design patterns	6–10 hours	Medium	Free / optional paid extras	Web tutorials	Extremely concise for design-pattern revision and interviews. [46]
Amazon official SDE topics and system-design prep	Company-specific prep	3–6 hours	Medium	Free	Official interview-prep pages/ videos	High ROI if you target Amazon, because it tells you exactly what Amazon expects. [47]
Flipkart SDE guide	Company-specific prep for Indian product interviews	2–4 hours	Medium	Free	Official PDF	One of the clearest public documents linking DSA, OOD, system design, distributed systems, and OS in a single company process. [48]

Recommended minimal stack by topic

If you want the fewest resources with the most coverage, use this stack:

- **DSA:** NeetCode 150 or Striver A2Z selected sections, then CSES for consolidation. [49]
- **OS:** OSTEP selected chapters only. [42]
- **DBMS:** SQLBolt first, then selective CMU 15-445 lectures on storage, indexing, transactions. [50]
- **Networking:** selective Stanford CS144 lectures or notes. [45]
- **OOP/OOD:** Refactoring.Guru plus 2–3 handwritten designs. [46]
- **System design:** Amazon's official prep material, Flipkart's guide, and engineering blog reading for concrete examples. Flipkart itself recommends reading engineering blogs and thinking in terms of distributed-system trade-offs. [51]

Suggested practice sets and platforms

For **coding rounds**, the best time-efficient progression is:

1. **NeetCode 150 core topics**
2. **CSES selected sections**
3. **Company-specific recent patterns** from LeetCode Discuss or official prep pages
4. **Timed mock contests / mock interviews**

This sequencing is better than doing random "company-tagged" lists too early because it builds reusable patterns first, then adds pressure and company nuance later. Recent Google India and Amazon India candidate reports strongly support structured pattern learning plus mock execution over random last-minute cramming. [52]

For **OOD / system design**, practice these prompts in writing and out loud:

- parking lot
- Splitwise
- chess / snake & ladder
- rate limiter
- URL shortener
- notification service
- feed/cache design
- distributed cache

They map well to what candidates recently reported at Amazon, Flipkart, Swiggy, and startup/product firms, and they force you to combine DSA, OOD, DBMS, OS intuition, and system-design trade-offs. [53]

Source notes

Highest-priority official sources

- Amazon official interview-prep pages for software development topics, university SDE roles, OA structure, and SDE-II loop/system-design expectations. [54]
- Microsoft official technical-interviewing and early-career hiring-tips pages, plus current India location/job pages. [55]
- Google official careers search/job pages showing DSA, software design, distributed systems, networking, and large-scale system expectations. [56]
- Flipkart's official SDE interview-prep guide PDF. [48]
- Current official/posting sources for India or India-linked product/backend roles at Amazon and Postman. [57]

Recent interview-experience sources

- LeetCode Discuss: Amazon SDE-1 fresher India, Google Bangalore / India L4-L5, plus related recent technical loop reports. [58]
- Medium: Flipkart SDE/GWC, Swiggy SDE-I, Swiggy SDE-II. [59]
- GeeksforGeeks recent interview/process pages for Microsoft, Infosys, Wipro, TCS. [60]

India hiring-trend sources

- TeamLease EdTech Career Outlook Report HY2 2025. [61]
- Naukri JobSpeak 2026 summaries and official blog index pages. [62]
- Reuters on India office leasing/GCC expansion; Zinnov and public GCC reports; PIB on GCC/talent framing. [63]

Learning-resource sources

- OSTEP. [42]
- CSES Problem Set. [41]
- CMU 15-445. [44]
- NeetCode roadmap and NeetCode 150. [64]
- Stanford CS144. [45]
- Refactoring.Guru. [46]
- TakeUForward / Striver A2Z. [65]
- SQLBolt. [43]

Assumptions used in this report

This analysis assumes the candidate has a **basic CS background**, is targeting **general SWE/SDE roles rather than kernel/compiler/network-specialist roles**, and is preparing for **India-based hiring as of July 2026**. Where a company's public materials were role-specific or level-specific, I generalized only to the cluster level and marked those estimates as synthesis rather than official policy. Recent market evidence suggests this cluster-based approach is more accurate than pretending all Indian hiring follows a single interview template. [66]

---

[1] [14] [16] [27] [47] [54] Amazon Software Development Interview Topics
https://amazon.jobs/content/en/how-we-hire/interview-prep/software-development-topics

[2] [57] Support Engineer, IN Amazon Pay - Job ID: 10376717
https://www.amazon.jobs/en/jobs/10376717/support-engineer-in-amazon-pay?utm_source=chatgpt.com

[3] Google | L4 | SWE 3 | India | Interview Experience & Journey | A Guide for DSA Dummies Like Me - Discuss - LeetCode
https://leetcode.com/discuss/post/6479658/google-l4-swe-3-india-offer-by-leethard-owg3/

[4] [11] [12] [37] (Updated) TCS NQT Syllabus 2026 & Exam Pattern
https://prepinsta.com/tcs-nqt/syllabus/?utm_source=chatgpt.com

[5] [23] [24] [26] [30] [34] [42] Operating Systems: Three Easy Pieces
https://pages.cs.wisc.edu/~remzi/OSTEP/?utm_source=chatgpt.com

[6] [61] [66] COR-HY2, July-Dec, 2025-V06
https://www.teamleaseedtech.com/pdf/COR-HY2-July-Dec-2025.pdf

[7] Software development engineer roles for students and grads
https://amazon.jobs/content/en/career-programs/university/sde

[8] [10] [28] [36] [51] SDE II Interview Prep
https://amazon.jobs/content/en/how-we-hire/sde-ii-interview-prep

9 60 Microsoft Interview Experience for SWE - GeeksforGeeks
https://www.geeksforgeeks.org/interview-experiences/microsoft-interview-experience-for-swe/

13 17 20 22 25 31 32 35 48 SDE
https://www.flipkartcareers.com/assets/flipkart_pdf/SDE.pdf

15 Infosys Interview Experience for System Engineer [2025] - GeeksforGeeks
https://www.geeksforgeeks.org/interview-experiences/infosys-interview-experience-for-system-engineer-2025/

18 Software Engineer, TPU Compiler, PhD, Early Careers
https://www.google.com/about/careers/applications/jobs/results/130170340289454790-software-engineer/?utm_source=chatgpt.com

19 29 Online assessment prep for SDE roles
https://amazon.jobs/content/en/how-we-hire/university/sde-oa

21 53 58 Amazon | SDE 1 Fresher | Accepted ✅ | India | Interview Experience - Discuss - LeetCode
https://leetcode.com/discuss/interview-experience/6361672/Amazon-or-SDE-1-Fresher-or-India-or-Interview-Experience/

33 39 64 Roadmap
https://neetcode.io/roadmap?utm_source=chatgpt.com

38 55 Technical interviewing | Microsoft Careers
https://careers.microsoft.com/v2/global/en/hiring-tips/technical-interviewing

40 65 Strivers A2Z DSA Course/Sheet - Crack Any FAANG or PBCs
https://takeuforward.org/dsa/strivers-a2z-sheet-learn-dsa-a-to-z?utm_source=chatgpt.com

41 CSES Problem Set - Tasks
https://cses.fi/problemset/?utm_source=chatgpt.com

43 50 SQLBolt - Learn SQL - Introduction to SQL
https://sqlbolt.com/?utm_source=chatgpt.com

44 CMU 15-445/645 :: Intro to Database Systems (Spring 2026)
https://15445.courses.cs.cmu.edu/?utm_source=chatgpt.com

45 CS 144: Introduction to Computer Networking
https://cs144.github.io/?utm_source=chatgpt.com

46 Design Patterns
https://refactoring.guru/design-patterns?utm_source=chatgpt.com

49 52 NeetCode 150 - Coding Interview Questions
https://neetcode.io/practice/practice/neetcode150?utm_source=chatgpt.com

56 Software Engineer, PhD, Early Career, AI/Machine Learning, 2026 Start — Google Careers
https://www.google.com/about/careers/applications/jobs/results/122258040807137990-software-engineer-phd-early-career-aimachine-learning-2026-start

59 Medium
https://medium.com/%40dronavallinandini/flipkart-interview-experience-software-development-engineer-1-43faefc3254b

62 Naukri's Official Blog
https://www.naukri.com/blog/

63 India's office leasing hits record as multinationals expand despite global uncertainty
https://www.reuters.com/world/india/indias-office-leasing-hits-record-multinationals-expand-despite-global-2026-07-06/?utm_source=chatgpt.com